

Chapter 8

Lorraine Gaudio

Version April 15, 2026



BOISE STATE UNIVERSITY

Contents

1	Overview	2
2	Set Up	3
2.1	Download and Save	3
2.2	Open an R Notebook	3
2.3	Load Data	4
3	Subsetting	6
3.1	☒ Positional Indexing	6
3.2	☒ Names Indexing	8
3.3	☒ Logical Mask Indexing	10
4	Extraction	16
4.1	[[Operator	16
4.2	\$ Operator	17
5	Subassignment (replacement)	19
5.1	Overwrite with <code>x[i, j] <- value</code>	19
5.2	Insertion	21
6	Code Chunk Options	24
7	Summary	25
8	Chapter Terms	26
9	☒ Practice Space	28
9.1	Task 1	28
9.2	Task 2	28
9.3	Task 3	29
9.4	Task 4	29
9.5	Task 5	30
9.6	Task 6	30
9.7	Task 7	30
9.8	Task 8	31
9.9	Task 9	31

1. Overview

Bracket indexing is one of the main ways you “point R” at the exact rows and columns you need. Real analysis rarely uses an entire dataset at once. Instead, analysts will select columns, filter rows, recode values or create new columns using subsetting, extraction, and insertion.

In this chapter’s guided activity you’ll practice those skills using the `field_sightings` dataset, a simulated bird-banding dataset where birds are recorded as **Tagged** or **Untagged** and tagged birds can have a band color. The data frame has two columns: `Status` and `Band_Color`. Then, you’ll use `Values_Transport_dplace.csv` in Practice Space to generalize your understanding, applying your new skills on a different dataset.

By the end of Chapter 8, you will be able to:

- ☐ Use a loaded CSV into R as a data frame.
- ☐ Apply the subsetting template to select data by ☐ positions, ☐ names, and ☐ logical mask.
- ☐ Explain missing value propagation in logical masks.
- ☐ Extract one column as “the thing itself” using `[[ ]]` or `$` so you can pass it into functions like `unique()` or `table()`.
- ☐ Experiment with subassignment methods to overwrite and insert values.
- ☐ Use code chunk options to manage long outputs in your R Notebook.

2. Set Up

To begin this guided activity, you'll want to

1. Download `field_sightings.csv` and `Values_Transport_dplace.csv` from Canvas and save them in the same folder as your course R Notebooks.
2. Open, format, and save an R Notebook in your course folder.

2.1 Download and Save

The final folder structure for this chapter will look like this:

```
- `Intro_to_R/`  
-- `chapter8_notes.Rmd`  
-- `chapter8_practice.Rmd`  
-- `Values_Transport_dplace.csv`  
-- `field_sightings.csv`
```

2.2 Open an R Notebook

To follow along Chapter 8, □ please open a new R Notebook in your course folder named `chapter8_notes.Rmd` and add a date to the YAML header. While your here, **include your name as the author!** See the example of a YAML below. Use this document to take notes and practice the chapter content.

1. Open RStudio → File → New File → R Notebook.
2. Save it in your course folder as: `chapter8_notes.Rmd`.
3. In your YAML header at the top:

```
---  
title: "Chapter 8 Practice: Bracket Indexing"  
author: "Your Name Here"  
date: "`r Sys.Date()`"  
output: html_notebook  
---
```

4. Remove the default content in the R Notebook and replace it with your notes and code as you work through the chapter.

5. Use headings as you work. At minimum, create section headings that match the chapter structure (e.g., `## Load Data`, `## Bracket Indexing`, `## Subsetting`, `## Extraction`, `## Insertion`. This creates a readable outline.
6. Memo as you go.
 - Outside chunks: write normal text.
 - Inside chunks: use `#` for any code comments (otherwise R tries to run the text).
 - If you hit an error: keep the broken line commented out and write a one-sentence note about what you changed to fix it.

2.3 Load Data

The `field_sightings.csv` dataset contains **simulated** (randomly generated) values. The dataset theme is based on the pretend situation where field biologists observed and recorded a population of species of birds. Bird banding is a scientific method used to track and study individual birds, providing valuable data on their movements, behavior, and population dynamics. In our senerio, most birds observed were “Untagged,” but a small number have been previously captured and marked with colored leg bands, “Tagged”. The color of the bands might provide information about the bird’s history, such as where it was banded or its age.

The simulated dataframe `field_sightings` has two columns:

- Status Describes whether the sighted bird is “Untagged” (the common case) or “Tagged” (the rare case).
- Band_Color If a bird is untagged, the value is “None.” If it is tagged, the value is one of several possible band colors.

□ **Goal:** Download the `field_sightings.csv` file from Canvas and save it in the *same* folder as your R Notebook. Then, load the data into R.

```
# □ Type this in your R Notebook
# 1) Confirm the CSV is in the same folder as your R Notebook
list.files()
```

□ **Goal:** Load `field_sightings.csv` into R using `read_csv()` and assign it to a variable called `feild_sightings`. Recall that you need to library `readr` before you can use `read_csv()`.

```
# □ Type this in your R Notebook
library(readr) # Load the readr package for reading CSV files
```

```
## Warning: package 'readr' was built under R version 4.5.2
```

```
field_sightings <- read_csv("field_sightings.csv")
```

```
## Rows: 4470 Columns: 2
## -- Column specification -----
## Delimiter: ","
## chr (2): Status, Band_Color
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

□ This code uses the `read.csv()` function to read the CSV file named “field_sightings.csv” and assigns the resulting data frame to a variable called `field_sightings`. It is normal to see messages with you load packages.

Feel free to click `field_sightings` in your Environment pane to view the data frame and get a sense of what it looks like.

3. Subsetting

Subsetting a smaller part of an object, typically using *positions* (integers), *names* (characters), or *logical mask* (TRUE/FALSE). In base R, this is done using the **single square bracket operator** `[ ( ] ]`.

The return output from a single square bracket operator is usually a container of the same type as the original object. A subset of a vector is still a vector and the subset of a data frame is often still a data frame. Data analysts use subsetting when filtering rows, selecting columns to keep, slicing a range, or pulling a rectangular block from a data frame.

3.1 Positional Indexing

Positional indexing is the most basic form of indexing in R, where you use numeric positions to select elements from a vector or data frame.

□ **Tip:** Positions start at 1 in R. This is different from other coding languages like python which starts indexing values at 0.

3.1.1 Vector

In Chapter 3, you practiced using the **indexing operator** with positional indexing to extract elements from vectors. Let's review that briefly before we move on to data frames.

You learned that the basic pattern for **positional indexing** is `x[i]`, where `x` is the vector and `i` is the index (or indices) of the elements you want to extract. A position used to identify elements in an object.

□ **Goal:** Practice using `[ ]` with positional indexing on vectors.

```
# □ Review
z <- c(10, 20, 30, 40, 50) # create a vector
```

```
# □ Review
z[1]
```

```
## [1] 10
```

□ The first element by position is 10.

```
# □ Review  
z[3]
```

```
## [1] 30
```

□ The third element by position is 30.

```
# □ Review  
z[2:4]
```

```
## [1] 20 30 40
```

□ The second, third and fourth elements by position is 20, 30, and 40.

3.1.2 Data Frame

The **indexing operator** (`[ ]`) is used to select or replace parts of many kinds of objects, not just vectors. For data frames, the common form is `df[rows, cols]` or `x[i, j]` where *i* is the **row index**, *j* is the **column index**. If you leave one side blank, the output includes all.

Rows and columns are coordinates:

- `df[1:5, ]` results in the first five rows, all columns
- `df[, 1:3]` shows all rows, first three columns
- `df[1, 2]` prints only one cell's value: first row, second column

Let's see this in practice with our `field_sightings` data frame.

□ **Goal:** Print the first six rows of the `field_sightings` data frame using the indexing operator.

```
# □ Type this in your R Notebook  
field_sightings[1:6, ]
```

Status	Band_Color
Untagged	None
Untagged	None
Untagged	None
Untagged	None
Untagged	None
Untagged	None

□ Shows first six rows, all columns.

3.1.3 head() Function Comparison

There is a function called `head()` that *basically* does the same thing as `field_sightings[1:6, ]` but is more concise and easier to read.

□ The SYNTAX:

```
head(x, n = 6L)
```

If you don't specify `n`, it defaults to 6, which means it will show the first 6 rows of the data frame. You can change this number to show more or fewer rows.

□ **Goal:** Print the first three rows of the `field_sightings` data frame using the `head()` function to check that the dataset loaded correctly.

```
# □ Type this in your R Notebook  
head(field_sightings, n = 3)
```

Status	Band_Color
Untagged	None
Untagged	None
Untagged	None

□ `head()` is commonly used to get a quick orientation of the dataset after loading.

`head()` does not do arbitrary extraction or subsetting. `head()` is a function that provides a convenient way to view the first few rows of a data frame.

Bracket indexing allows for flexible and specific subsetting of the data frame based on row *and* column indices. `[` works on almost everything and supports rich indexing and is commonly used as part of an algorithm. You will use `[` when you need subsetting by columns, conditions, and arbitrary rows when you are writing analysis code.

3.2 Names Indexing

You can also use **names indexing** (also referred to as character indexing) to select columns by their names instead of their positions. This often makes more readable code and is less error-prone, especially when the order of columns might change or when you have many columns.

3.2.1 Vector

In Chapter 3, you created the `nerdy_palette` vector and then used names indexing to return the hexadecimal color code.

```
# □ Review
# Create nerdy_palette
nerdy_palette <- c(
  "Octarine"    = "#7F00FF",
  "TARDIS Blue" = "#003B6F",
  "Rebeccapurple" = "#663399",
  "Hoolooovoo"  = "#4F97A3",
  "Cosmic Latte" = "#FFF8E7",
  "Vantablack"  = "#000000",
  "Matrix Green" = "#03A062",
  "Source Engine Magenta" = "#FF00FF",
  "Command Red"  = "#CC0000",
  "Impossible Green" = "#4D9E68"
)

# Use names indexing to return the hex code for Octarine
nerdy_palette["Octarine"]
```

```
## Octarine
## "#7F00FF"
```

□ The output shows the hexadecimal color code for “Octarine” retrieved from the `nerdy_palette` vector using its name.

3.2.2 Data Frame

A common use of names indexing is to select a column from a data frame by its name.

□ **Goal:** Use names indexing to return the band colors in the `field_sightings` data frame.

Methods A:C show different ways to subset a column with `[]`.

Method A

□ The SYNTAX: `x[i]`

```
# □ Type this in your R Notebook
field_sightings["Band_Color"]    # 1-column data frame
```

□ A 1-column data frame is printed showing all row values in the column `Band_Color` the `field_sightings` data frame. It is a subset of one column, keeping that data-frame structure.

Method B

□ The SYNTAX: `df[rows, cols]`

```
# □ Type this in your R Notebook
# Example B
field_sightings[, "Band_Color"] # usually the column VECTOR
```

□ A vector is printed showing all values from the column `Band_Color` the `field_sightings` data frame. The result is a subset from one column and returns a vector, values only.

Method C

□ The SYNTAX: `x[i, j, drop = FALSE]`

```
# □ Type this in your R Notebook
# Example C
field_sightings[, "Band_Color", drop = FALSE] # 1-column data frame
```

□ A 1-column data frame is printed showing all row values in the column `Band_Color` the `field_sightings` data frame. It is a subset of one column, keeping that data-frame structure.

□ **Tip:** Students often forget to put the column name in quotes when using names indexing. If you forget the quotes, R will look for an object named `Band_Color` in your environment and return an error if it doesn't find one. Always remember to put column names in quotes when using **names indexing** with `[ ]`.

3.3 ☒ Logical Mask Indexing

A **logical mask** is the logical vector you use to index.

3.3.1 Vector

In Chapter 4, you practiced logical indexing with a vector.

□ Create a new vector called `smallerthat` contains only values in `number_list` that are less than or equal to 20.

```
# □ Type this in your R Notebook
# Create the sample vector
number_list <- c(10, 25, 5, 30, 15, 40)
```

```
# □ Check
print(number_list)
```

```
## [1] 10 25 5 30 15 40
```

□ The `number_list` vector contains the values 10, 25, 5, 30, 15, and 40.

```
# □ Type this in your R Notebook
# Example 1
# Step 1: create the logical "mask"
mask <- number_list <= 20
```

```
# □ Check
print(mask)
```

```
## [1] TRUE FALSE TRUE FALSE TRUE FALSE
```

□ This prints the mask. The mask itself is not a subset of the original vector. The logical vector indicates which values in `number_list` are less than or equal to 20.

```
# □ Type this in your R Notebook
# Example 1
# Step 2: use the logical vector inside []
smaller <- number_list[mask]

# □ Check
print(smaller)
```

```
## [1] 10 5 15
```

□ This keeps only the values where the condition is TRUE (10, 5, 15).

You can also pass the mask directly into the indexing operator without storing it in a variable first.

```
# □ Type this in your R Notebook
# Example 2
smaller <- number_list[number_list <= 20]
```

□ Both example 1 and example 2 are **conditional subsetting** or filtering by using a logical condition. These examples use the **less than or equal to (<=) relational operator**. The **logical mask** is the result of the comparison (`number_list <= 20`).

`[]` uses it as a filter. **Logical mask indexing** is a form of indexing in R where you use a logical vector (a “mask” of TRUE/FALSE values) to select elements from a vector or rows from a data frame.

R keeps the values where the mask is TRUE and drops the values where the mask is FALSE.

Whether you create an object to store the logical mask (example 1) or pass the logical condition directly into the indexing operator (example 2), the result is the same: you get a subset of the original vector that meets the specified condition.

Let's see this in action in a dataframe.

□ **Review:** Chapter 3 introduced comparison operators.

Operator	Meaning	Example
>	greater than	<code>mpg > 20</code>
<	less than	<code>mpg < 15</code>
==	equal to	<code>am == 1</code>

Operator	Meaning	Example
>=	greater <i>or</i> equal	mpg >= 25
<=	less <i>or</i> equal	mpg <= 15
!	not	!(gear %in% c(3,4))
!=	not equal to	gear != 4

3.3.2 Data Frame

Logical mask indexing works the same way for data frames, except that the logical mask *usually* goes in the **rows** position of `df[rows, cols]`.

The mask can be TRUE/FALSE/NA for each row. When you use the mask in `df[mask, ]`, R keeps only the rows where the mask is TRUE and NA if there are missing values.

□ **Goal::** Keep only rows where the bird is tagged using `[]`.

```
# □ Type this in your R Notebook
# Step 1: create the logical mask (one TRUE/FALSE per row)
mask_tagged <- field_sightings[, "Status"] == "Tagged"
```

□ The `==` is a comparison operator that checks if the value in the Status column is equal to “Tagged”. A single `=` is used for assigning values inside some function calls (like `na.rm = TRUE`) and naming arguments. It is not used to test equality.

```
# □ Check
head(mask_tagged)
```

```
##      Status
## [1,] FALSE
## [2,] FALSE
## [3,] FALSE
## [4,] FALSE
## [5,] FALSE
## [6,] FALSE
```

□ This prints a logical mask the same length as the number of rows in `field_sightings`. Each TRUE means this row has `Status == 'Tagged'`.

```
# □ Type this in your R Notebook
# Step 2: use the logical mask inside [] to subset rows
tagged_only <- field_sightings[mask_tagged, ]

# Alternative: pass the logical mask directly into []
```

```
# tagged_only <- field_sightings[field_sightings[, "Status"] == "Tagged", ]

# □ Check
head(tagged_only)
```

Status	Band_Color
Tagged	Orange
Tagged	Silver
Tagged	Green
Tagged	Black
Tagged	Silver
Tagged	Blue

□ This subsets the data frame by keeping only rows where the mask is TRUE.

```
# □ Check
nrow(field_sightings)      # how many rows in the original data frame?
```

```
## [1] 4470
```

```
nrow(tagged_only)        # how many tagged rows?
```

```
## [1] 325
```

```
sum(mask_tagged, na.rm=TRUE) # how many TRUE values in the mask?
```

```
## [1] 300
```

□ The result is a smaller data frame containing only the tagged birds. There are NA values, so we needed to use `na.rm = TRUE` to get an accurate count only the TRUE values. We can tell the mask contains NA values because `nrow(tagged_only)` (325) is larger than `sum(mask_tagged, na.rm=TRUE)` (300).

Logical mask indexing is a powerful way to subset data frames based on conditions, but missing values can make the results harder to interpret. When we compute `mask_tagged <- field_sightings[, "Status"] == "Tagged"`, any row where Status is NA produces NA (because `NA == "Tagged"` is unknown). That's **missing value propagation**! This is where missing values carry through comparisons as NA.

When we then subset with `tagged_only <- field_sightings[mask_tagged, ]`, R keeps rows where the mask is TRUE, but it does not treat NA as FALSE. Rows with NA in the mask can appear in the subset rows, which is why `tagged_only` has 325 rows even though there are only 300 TRUE values in the mask.

3.3.3 which() Function Helper

The function `which()` is a subsetting helper that turns a logical masks into row indices. Using `which()` is a strategy to handle potential NA values in your data. When the test could include NA, wrap it in `which()` to drop unknowns.

□ **Goal:** Use `which()` to subset the `field_sightings` data frame to keep only rows where the bird is tagged, while safely handling any potential NA values in the “Status” column.

You saw the function `which()` in Chapter 6. This function converts TRUE positions into numeric indices.

□ *Type this in your R Notebook*

```
# Pass logic through which() to get the row indices where Status is "Tagged"
row_idx_tagged <- which(field_sightings[, "Status"] == "Tagged")
```

```
## Alternative: Create mask_tagged then pass it to which() to get row indices
# mask_tagged <- field_sightings[, "Status"] == "Tagged"
# row_idx_tagged <- which(mask_tagged)
```

```
# Use the row indices to subset the data frame by rows that are tagged
tagged_only <- field_sightings[row_idx_tagged, ]
```

□ *Check*

```
nrow(tagged_only)
```

```
## [1] 300
```

```
head(tagged_only, 10)
```

Status	Band_Color
Tagged	Orange
Tagged	Silver
Tagged	Green
Tagged	Black
Tagged	Silver
Tagged	Blue
Tagged	Green
Tagged	Black
Tagged	White
Tagged	Red

□ `which()` returns the indices of the TRUE elements and omits positions where the test result is FALSE or NA values for the logical vector passed in. If the logical test can produce NA, `which()` drops unknowns.

In conclusion, subsetting (`[]`) returns a piece of the original object, often preserving structure. When you need a smaller data frame, *subset*. If you need values within another function, you'll need to *extract*. In the next section, you will learn how extraction returns one component's contents.

4. Extraction

Extraction means pulling out a single component. When you want the column values themselves as a vector, operators `[[ ]]` and `$` are the clearest extraction approach. Extraction pulls out *a single component* (one column or one list element) so you can use it inside a function or calculation. Data analysts use extracted objects in a specific part of more complex code that creates a plot or summary.

4.1 `[[` Operator

There is another operator called the **double square bracket operator** (`[[ ]]`). `[ ]` is the general subsetting operator. `[[ ]]` extracts exactly one component. `[ ]` returns a subset of the same type as the original object. `[[ ]]` returns a single element inside the object (the “thing itself”), not a same-type subset. `[[ ]]` is extraction because it *pulls out* one component as the vector.

Operator	Meaning
<code>[]</code>	Subsetting operator. Returns a subset of the same type as the original object. Can return multiple elements.
<code>[[]]</code>	Extraction operator. Returns a <i>single element</i> inside the object and <i>passed</i> into functions.

□ **Goal:** Use the `[[` operator to extract the “Band_Color” column from the `field_sightings` data frame and then use it inside the function `unique()`

□ The SYNTAX: `x[[i]]`

Step 1

Extract a column as a vector

```
# □ Type this in your R Notebook
field_sightings[["Band_Color"]] # the column VECTOR
```

□ The result is an extraction from one column and returns a vector, values only.

Step 2

Use the extracted vector inside the function `unique()`

```
# □ Type this in your R Notebook
unique(field_sightings[["Band_Color"]])
```

```
## [1] "None" "Orange" "Silver" "Green" "Black" NA "Blue" "White"
## [9] "Red" "Yellow"
```

□ This shows the unique band colors in the `field_sightings` data frame. The `[[` operator allows you to extract the column as a vector, which is what `unique()` needs to work properly.

4.2 \$ Operator

There is another operator called the **dollar sign operator** (\$) that is used to extract columns from a data frame by name. It is a convenient shorthand for selecting a single column by name when you type the name literally.

□ The SYNTAX:

```
dataframe$column_name
```

□ **Goal:** Extract the `Band_Color` column using \$.

```
# □ Type this in your R Notebook
field_sightings$Band_Color
```

□ A vector is printed showing all element values from the column `Band_Color` from the `field_sightings` data frame. It is an extraction from one column and returns a vector, values only.

You will likely use the \$ operator often. The \$ operator is more concise and easier to read for common uses cases. However, it only works for selecting one column at a time and can't use an assigned variable stored in the environment.

If we try to use a variable to specify the column name with the \$ operator, it will not work because \$ does not evaluate the variable name. `[[ ]]` can use a variable and is safer in programs.

□ **Goal:** Use a stored *object* name to specify the column name and try to extract the column using both `[[` and \$ operators.

```
# □ Type this in your R Notebook
# Example 1
col_name <- "Band_Color"
field_sightings[[col_name]] # □ works
```

□ The `[[` operator allows you to use a variable name from your environment to specify the column name, which is why it works in this case. The result is an extraction from one column and returns a vector, values only.

```
# □ Type this in your R Notebook
# Example 2
col_name <- "Band_Color"
field_sightings$col_name    # □ looks for a column literally named "col_name"
```

```
## Warning: Unknown or uninitialised column: `col_name`.
```

```
## NULL
```

□ The `$` operator looks for a column literally named “col_name” and finds none, resulting in `NULL`.

So far, you’ve learned two different moves:

- Subset: `field_sightings["Band_Color"]` (data frame)
- Extract: `field_sightings[["Band_Color"]]` or `field_sightings$Band_Color` (values)

Next, we’ll learn how to change values inside an existing object using replacement methods.

5. Subassignment (replacement)

Subassignment means replacing part of an object (vector, matrix, data frame, list, etc.) with new values using indexing. It is an umbrella term for *assigning* into part of an object using an index on the left of `<-`.

In this chapter, you'll use three core replacement patterns for data frames:

1. Use `x[i, j] <- value` to change specific rows and columns.
2. Use `x[["..."]] <- value` to replace/add one column (works when the column name is stored in a variable).
3. Use `x$name <- value` to replace/add one column (when the column name is typed literally).

□ **Indexing on the left of `<-` edits the object you assign into**

5.1 Overwrite with `x[i, j] <- value`

Overwrite is an informal type of replacement where you change existing values.

□ The SYNTAX:

```
x[i, j] <- value
```

Notice that indexing is located on the left side of an assignment operator (`<-`) to change specific values in a data frame.

What if all the tagged birds in the `field_sightings` data frame were raised as part of a breeding program and then reintroduced into an area where the species no longer existed? This means all the untagged birds were hatched in the wild. Suppose we decide to rename the element value "Untagged" to "Wild Hatch" in the `Status` column to communicate this information more clearly.

□ **Goal:** Replace "Untagged" with "Wild Hatch" in `Status`.

Step 1

Make a copy of the data frame.

□ **Tip:** Always make a copy of your data frame before you overwrite values. This way, if something goes wrong, you still have the original data intact.

```
# □ Type this in your R Notebook
reintroduction <- field_sightings      # copy
```

□ We will overwrite values using the copy dataframe, reintroduction.

Step 2

Get row indices with `which()` to identify the rows where Status is “Untagged”. `which()` returns the row positions where the test is TRUE and ignores positions where the test is FALSE or NA. Use the dataframe that you want to overwrite values in, not the original data frame.

```
# □ Type this in your R Notebook  
indices_untagged <- which(reintroduction[, "Status"] == "Untagged")
```

Step 3

Overwrite using `df[rows, "col"] <- value`

```
# □ Type this in your R Notebook  
reintroduction[indices_untagged, "Status"] <- "Wild Hatch"
```

Step 4

Verify that the values have been changed.

```
# □ Check  
unique(reintroduction[, "Status"]) # modified data frame
```

Status

Wild Hatch
Tagged
NA

```
unique(field_sightings[, "Status"]) # original data frame
```

Status

Untagged
Tagged
NA

□ The Status column in the reintroduction data frame has been modified to replace “Untagged” with “Wild Hatch”.

5.2 Insertion

Insertion is an informal type of replacement where you add something new (most commonly a new column).

`x[[...]] <- value` and `x$name <- value` replace (or create) one component. For data frames, that's typically a single column.

Suppose we decide to add a new column called `Tag_Flag` that indicates whether the bird is tagged or not. It is common to use binary columns to create figures or run analyses. We want to set the value to 1 for tagged birds and 0 for untagged birds so we can later create a visualization of how many tagged vs. untagged birds we have.

5.2.1 `[[` Method

□ **Goal:** Add a new column (a flag) using a column name stored in a variable.

Step 1

Make a copy of the data frame.

```
# □ Type this in your R Notebook
reintroduction2 <- field_sightings
```

Step 2

Store the new name of your column a variable

```
# □ Type this in your R Notebook
new_col <- "Tag_Flag"
```

Step 3

Add the new column using `[[ ]]` <-

```
# □ Type this in your R Notebook
# Start everyone as 0
reintroduction2[[new_col]] <- 0          # <- Uses [[ ]]

# Change to 1 where Status is Tagged
idx_tagged <- which(reintroduction2[, "Status"] == "Tagged")
reintroduction2[idx_tagged, new_col] <- 1
```

Step 4

Verify that the new column has been added and values are correct.

```
# □ Check
head(reintroduction2)
```

Status	Band_Color	Tag_Flag
Untagged	None	0
Untagged	None	0
Untagged	None	0
Untagged	None	0
Untagged	None	0
Untagged	None	0

5.2.2 \$ Method

If you are typing the column name directly (not stored in a variable), you can use \$ to add the column.

Step 1

Make a copy of the data frame.

```
# □ Type this in your R Notebook
reintroduction3 <- field_sightings
```

Step 2

Add the new column using \$ <-

```
# □ Type this in your R Notebook
# Start everyone as 0
reintroduction3$Tag_Flag <- 0          # <- Uses $

# Change to 1 where Status is Tagged
idx_tagged <- which(reintroduction3[, "Status"] == "Tagged")
reintroduction3[idx_tagged, "Tag_Flag"] <- 1
```

Step 3

Verify that the new column has been added and values are correct.

```
# □ Check
head(reintroduction3)
```

Status	Band_Color	Tag_Flag
Untagged	None	0
Untagged	None	0

Status	Band_Color	Tag_Flag
Untagged	None	0
Untagged	None	0
Untagged	None	0
Untagged	None	0

□ Use `[[ ]]` when the column name is stored in a variable (like `new_col`). Use `$` when you are typing the column name literally.

6. Code Chunk Options

You probably noticed that some code chunk examples in this chapter resulted in long vector outputs. These can become difficult to read and navigate. You might find them particularly annoying when reviewing your HTML report. This final section explains how to manage long outputs in your R Notebook.

To make your R Notebook more readable, you can use the `results = 'hide'` option in your code chunks to hide the output of certain chunks, especially those that produce long outputs.

Alternatively, you can use `eval = FALSE` to prevent the code from running at all, which is useful for code that is meant to be illustrative rather than executed, such as when there is an error that you want to discuss without actually running the code.

Below are some examples of how to use these options in your R Notebook. Try out the code chunk settings in your notebook to see how they work!

```

```{r, echo=TRUE, results='hide'}
Show code, hide long output
field_sightings[["Band_Color"]]
```

```{r, echo=FALSE, message=FALSE, warning=FALSE}
Run, but don't show code/messages/warnings
field_sightings$col_name
```

```{r, eval=FALSE, echo=TRUE}
Show code, but do not run it
field_sightings$Band_Color
```

```

- Experiment with these options in your R Notebook to find the best way to manage long outputs above to make your notes and practice reports more readable. Knit your notebook to HTML to see how it looks, then adjust the options and other formatting and re-knit to make sure your report is professional and ready to share with your research collaborators.

7. Summary

In Chapter 8, you learned three core indexing patterns in base R for working efficiently with data frames.

First, you practiced subsetting with `[ ]` to pull out a smaller piece using the `df[rows, cols]` pattern (for example, selecting a range of rows, selecting a single column as a vector with `df[, "Band_Color"]`, or keeping it as a 1-column data frame with `df["Band_Color"]`).

Second, you practiced extraction with `[[ ]]` and `$` to get the column itself as a vector so it can be fed directly into functions, and you learned the key practical difference: `df[[col_name]]` works when the column name is stored in a variable, while `df$ColName` only works when you type the column name literally (so `df$col_name` does not mean “use the column named in `col_name`”).

Third, you practiced subassignment/replacement by putting indexing on the left side of `<-` to overwrite specific cells (e.g., `df[rows, "col"] <- value`) and to add or replace entire columns (e.g., `df[[new_col]] <- value`, which creates the column if it doesn't exist).

You also learned why logical filtering can break in subtle ways due to NA propagation: comparisons like `df[, "Status"] == "Tagged"` produce NA where Status is NA, and subsetting with `df[mask, ]` can behave unexpectedly; a reliable fix is `which(df[, "Status"] == "Tagged")`, because `which()` keeps only TRUE positions and drops FALSE and NA.

In the next chapter, you'll extend filtering by combining conditions with `&`, `|`, and `%in%`.

8. Chapter Terms

Column index: The index used to select columns in a data frame or matrix (the second slot inside brackets: `x[, j]`).

Conditional Subsetting (Logical Subsetting): Filtering an object by using a logical condition inside `[` (e.g., `df[df$cyl != 4, ]`), where TRUE keeps an element/row and FALSE drops it. This contrasts with negative indexing, which excludes by position rather than by a condition.

Dollar Sign Operator (`$`): An extraction (and replacement) operator used to access a single named component of a recursive object. Most commonly a data frame column (`df$col`) or a list element (`lst$name`). `$` requires a literal name (not a name stored in a variable), returns NULL if the name doesn't exist, and by default may do partial matching (which can surprise you).

Double Square Bracket Operator (`[[`): An extraction operator used to pull out one single element from a list (or one column from a data frame) and return the element itself, not a same-type subset. Unlike `[`, which can return multiple elements and preserves the container type, `[[` is for “give me the one thing inside” (e.g., a vector inside a dataframe).

Equal To (`==`): A relational operator that tests equality and returns TRUE where values are equal and FALSE where they are not (element-by-element for vectors).

Extraction: Retrieving a single component from an object—such as one list element or one data-frame column—so you get the “thing itself” (often a simplified result like a vector). Extraction is commonly done with `[[ ]]` (extract one element by position or name) or `$` (extract one column by name).

Indexing Operator: The square-bracket indexing operator `[` or `[[` are used to extract or replace parts of an object in R (for example, `x[i]`). For vectors, it selects one or more elements and returns the same kind of object as `x` (a vector).

Insertion: A common type of subassignment where you add something new to an object—most often a new column in a data frame. Example: `df[["Tag_Flag"]] <- 0` (creates the column if it doesn't exist yet).

Less Than or Equal To (`<=`): A relational operator that returns TRUE where the left side is less than or equal to the right side, and FALSE otherwise.

Logical Indexing: Indexing a vector using a logical vector of TRUE/FALSE values (for example, `x[x > 0]`) so R keeps the elements where the index is TRUE and drops those where it is FALSE. If the logical index is a different length, R may recycle it, which can lead to surprising results.

Missing Value Propagation: R's behavior where an NA (unknown value) “carries through” computations and comparisons. If an operation can't be determined because of missing data, the result is often NA (unknown) instead of TRUE or FALSE. For example, `NA == "Tagged"` returns NA. If you use a logical mask that contains NA values for subsetting (e.g., `x[mask]` or `df[mask, ]`), R does

not treat NA as FALSE, so the subset may include NA values (or NA rows), which can change counts and results.

Names Indexing: Indexing a named vector using character strings that match element names (for example, `x[c("Octarine", "TARDIS Blue")]`). With `[]`, names are matched exactly; any name that doesn't exist returns NA for that position.

Overwrite: A common type of subassignment where you change existing values (you replace what is already there). Example: `df[rows, "Status"] <- "Wild Hatch"`.

Positional Indexing: Indexing a vector using numeric positions (for example, `x[1]` for the first element or `x[c(1, 3)]` for the first and third). Positions are based on the current order of the vector.

Row index: The index used to select rows in a data frame or matrix (the first slot inside brackets: `x[i, ]`).

Subassignment: Replacing part of an object by putting an index on the left side of `<-`. R changes only the indexed part of the object. Examples: `x[i] <- value`, `df[rows, "col"] <- value`, `df[[col]] <- value`, `df$col <- value`.

Subsetting: Selecting a smaller piece of an R object using indices (positions, names, or a logical mask). With data frames, subsetting most commonly uses `[]` in the form `df[rows, cols]` and typically preserves the object's structure (e.g., you still get a data frame when you subset rows and columns).

9. ☒ Practice Space

This Practice Space uses a new dataset to help you generalize your learning. While the column names differ, the indexing patterns are the same. You will use `Values_Transport_dplace.csv` using the columns `id`, `name`, `sub_case`, and `year`.

Open a new R Notebook and save it in your course folder as: `chapter8_practice.Rmd`. Edit the YAML so that it includes the title “Chapter 8 Practice”, your name as the author, and the date. Remove the default content in the R Notebook. Create heading for each task.

Work through each task, creating learning memos as you go. Outside of the code chunks, write a few sentences reflecting on what you learned from each task and how it connects to the concepts in this chapter.

If you use an LLM to help you understand the code, document your use transparently. Do not use LLMs to complete the tasks for you. The goal is to practice writing the code yourself and to reflect on your learning process.

9.1 Task 1

Load the CSV with `readr::read_csv()`

☐ **Goal:** Load `Values_Transport_dplace.csv` into R using `read_csv()` and assign it to a variable called `values_transport`.

```
# ☐ Type this in your R Notebook, Fill in the blanks
# 1) Confirm the CSV is in the same folder as your R Notebook
list.files()

# 2) Load required package, then read the CSV
library(____)

_____ <- read_csv(__Values_Transport_dplace.csv__)
```

9.2 Task 2

☐ Positional subsetting (rows/cols by index number)

☐ **Goal:** Create `first_10_basic` containing the first 10 rows and columns 2 through 5 of `values_transport`. Use `df[rows, cols]`

□ *Type this in your R Notebook, Fill in the blanks*

```
first_10_basic <- _____[____, ____]
```

□ How many rows and how many columns does first_10_basic have?

9.3 Task 3

□ Names/character subsetting (choose specific columns by name)

□ **Goal:** Create id_name_year containing only the columns: id, name, and year.

□ *Type this in your R Notebook, Fill in the blanks*

```
_____ <- values_transport[, c(____, ____, ____)]
```

□ *Check*

```
names(id_name_year)
```

```
head(id_name_year)
```

□ The id_name_year data frame contains only the columns id, name, and year from the original values_transport data frame.

9.4 Task 4

□ Logical mask subsetting

□ **Goal:** Subset rows where year greater than or equal to 1900 using which() to ensure not NA problems.

□ *Type this in your R Notebook, Fill in the blanks*

```
idx_1900 <- _____(values_transport[, ____] ____ 1900)
```

```
values_1900 <- values_transport[_____, ]
```

□ *Check*

```
nrow(values_transport) # original number of rows
```

```
nrow(values_1900)      # number of rows where year >= 1900
```

□ The values_1900 data frame contains only the rows from values_transport where the year column is greater than or equal to 1900. The number of rows in values_1900 should be less than or equal to the number of rows in the original values_transport data frame.

□ How many entries have a year of 1900 or later?

9.5 Task 5

☐ Explain Missing Value Propagation

☐ **Goal:** In 3–5 sentences, explain what **Missing Value Propagation** means when you create a logical mask like:

```
mask <- values_transport[, "year"] >= 1900
```

If there were NA values in the year column. What will R return when a comparison involves NA (example: NA >= 1900)? Why can NA values in a logical mask cause confusing subsetting results with [mask,]? How does which() help to avoid this issue?

9.6 Task 6

☐ Extraction

☐ **Goal:** Extract the year column as a vector using [[]], then run table().

```
# ☐ Type this in your R Notebook, Fill in the blanks  
____(values_transport__"year"__)
```

☐ Comment on the output of unique(year_vector). What do you notice about possible values in the year column in the values_transport data frame?

9.7 Task 7

☐ Overwrite

Scenario: You found four data-entry errors in the year column. After reviewing field notes, you confirm these corrections:

- -621 should be 1821
- -1750 should be 1750
- 110 should be 1910
- 1292 should be 1892

☐ **Goal:** Use subassignment with df[rows, "col"] <- value to overwrite the incorrect year values in the copy of values_transport data frame called transport_year_clean.

☐ Type this in your R Notebook, Fill in the blanks

Step 1: Make a copy of the data frame

```
_____ <- _____
```

Step 2: Identify row indices for each incorrect value

```
idx_621 <- _____(transport_year_clean[, "year"] == -621)
```

```
idx_1750 <- _____(transport_year_clean[, "year"] == -1750)
```

```
idx_110 <- _____(transport_year_clean[, "year"] == 110)
```

```
idx_1292 <- _____(transport_year_clean[, "year"] == 1292)
```

Step 3: Overwrite the incorrect values with the correct ones

```
transport_year_clean[_____, "year"] <- 1821
```

```
transport_year_clean[_____, "year"] <- 1750
```

```
transport_year_clean[_____, "year"] <- 1910
```

```
transport_year_clean[_____, "year"] <- 1892
```

☐ Check

```
unique(transport_year_clean[["year"]])
```

☐ Explain why we (1) made a copy first and (2) used which() to find row indices before overwriting.

9.8 Task 8

☐ Save your workspace (chapter8_practice.RData) for submission. This file should contain:

- values_transport (original data frame loaded from CSV)
- first_10_basic (subset with first 10 rows and columns 2-5)
- id_name_year (subset with only id, name, year columns)
- values_1900 (subset with rows where year >= 1900)
- transport_year_clean (data frame with corrected year values)
- mask (logical vector for year >= 1900)

9.9 Task 9

☐ **Goal:** Create a well-formatted HTML report for submission.

Step 1: Sweep your Environment (start clean).

Click the ☐ **broom icon** in the Environment pane to clear objects.

Step 2: Run everything top-to-bottom.

Use **Run** → **Run All** to execute every chunk in order.

Step 3: Save and submit.

After your notebook runs without errors, save your notebook (.Rmd) and knit to HTML (.HTML) for submission to Canvas, as directed.

□ **Tip:** After you knit, look in your course folder for two files with the same name. One file ends in .Rmd and one ends in .html. The HTML usually has a internet browser icon. You can check which is which by double clicking to open them. One opens in R and the other usually opens in a web browser. Upload the .html file. Before submission, check that the uploaded file ends in .html.